

Curso de Go

Aula 18 - Middlewares e Boas Práticas REST em Go
Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **BRAZUCA**





Nessa Aula nos Vamos Ver

- **O que são Middlewares**
- **Como implementar Middlewares em Go**
- **Encadeamento de Middlewares**
- **Logging de requisições**
- **Autenticação básica**
- **Princípios REST**
- **Métodos HTTP e códigos de status**
- **Organização de uma API REST em Go**
- **Parte Prática**



O que é um Middleware?

- Um Middleware é uma função executada entre a chegada da requisição HTTP e o processamento realizado pelo Handler.
- Fluxo tradicional: Cliente → Servidor → Handler → Resposta
- Com Middleware: Cliente → Middleware → Handler → Resposta
- Exemplos de utilização:
 - Logging
 - Autenticação
 - Controle de acesso
 - CORS (Cross-Origin Resource Sharing)
 - Validação
 - Recuperação de erros
 - Medição de desempenho
- O Middleware evita repetir o mesmo código em vários endpoints.



Estrutura de um Middleware em Go

- No pacote `net/http`, podemos criar Middlewares utilizando `http.Handler`.

```
func middleware(next http.Handler)
http.Handler {

    return http.HandlerFunc(func(w
http.ResponseWriter,
    r *http.Request) {

        fmt.Println("Middleware
executado")

        next.ServeHTTP(w, r)
    })
}
```



A chamada:

`next.ServeHTTP(w, r)`

permite que a requisição continue sendo processada.



Utilizando o Middleware

- **Primeiro passo : criamos nosso Handler**

```
func home(w http.ResponseWriter, r
*http.Request) {
    fmt.Fprintln(w, "API funcionando")
}
```

- **Depois aplicamos o Middleware:**

```
func main() {

    handler := http.HandlerFunc(home)

    http.Handle("/",
middleware(handler))

    http.ListenAndServe(":8080", nil)
}
```



Ao acessar

`http://localhost:8080`

o Middleware será executado antes do Handler home.



Encadeamento de Middlewares

- Uma aplicação pode possuir vários Middlewares.
- **Exemplo: Cliente → Logging → Autenticação → Validação → Handler → Resposta**

```
phandler := http.HandlerFunc(home)
```

```
handlerFinal :=  
    logging(  
        autenticacao(  
            handler,  
        ),  
    ),  
}
```



- Cada Middleware possui uma responsabilidade específica.
- Essa separação melhora a organização do código.



Middleware de Logging

- **Logging permite registrar informações sobre as requisições recebidas.**
- **Exemplo:**

```
func logging(next http.Handler) http.Handler {  
  
    return http.HandlerFunc(func(w http.ResponseWriter,  
        r *http.Request) {  
  
        log.Println(  
            r.Method,  
            r.URL.Path,  
        )  
  
        next.ServeHTTP(w, r)  
    })  
}
```

- **Podemos Registrar:**
 - **Método HTTP**
 - **Endpoint acessado**
 - **Data e hora**
 - **IP do cliente**
 - **Tempo de processamento**
 - **Código HTTP retornado**

Logs são fundamentais para monitoramento e diagnóstico de problemas!



Medindo o Tempo da Requisição

- Também podemos medir quanto tempo uma requisição levou.

```
func logging(next http.Handler) http.Handler {  
  
    return http.HandlerFunc(func(w http.ResponseWriter,  
        r *http.Request) {  
  
        inicio := time.Now()  
  
        next.ServeHTTP(w, r)  
  
        log.Printf(  
            "%s %s %v",  
            r.Method,  
            r.URL.Path,  
            time.Since(inicio),  
        )  
    })  
}
```

- Exemplo de log:

GET /products 2.31ms

Isso ajuda a identificar endpoints lentos e problemas de desempenho.



Autenticação HTTP Basic

- HTTP Basic Authentication envia usuário e senha através do cabeçalho: Authorization
- Em Go podemos recuperar essas informações com:

```
usuario, senha, ok := r.BasicAuth()
```

- Exemplo:

```
if !ok {  
    http.Error(  
        w,  
        "Não autorizado",  
        http.StatusUnauthorized,  
    )  
    return  
}
```

Basic Auth é simples e útil para demonstrações e alguns cenários controlados, mas deve ser utilizada sobre HTTPS, pois as credenciais não são criptografadas pelo próprio mecanismo Basic.



Middleware de Autenticação

- Podemos transformar a autenticação em Middleware:

```
func auth(next http.Handler) http.Handler {  
  
    return http.HandlerFunc(func(w http.ResponseWriter,  
        r *http.Request) {  
  
        usuario, senha, ok := r.BasicAuth()  
  
        if !ok ||  
            usuario != "admin" ||  
            senha != "1234" {  
  
            w.Header().Set(  
                "WWW-Authenticate",  
                `Basic realm="API"`,  
            )  
  
            http.Error(  
                w,  
                "Não autorizado",  
                http.StatusUnauthorized,  
            )  
            return  
        }  
  
        next.ServeHTTP(w, r)  
    })  
}
```

- Assim, qualquer endpoint pode ser protegido pelo mesmo Middleware.
- Em aplicações reais, senhas não devem ficar diretamente no código.



O que é REST?

- REST significa: Representational State Transfer
- É um estilo arquitetural utilizado para construção de serviços e APIs Web.
- Uma API REST normalmente trabalha com recursos.
- Exemplos:

```
/products  
/users  
/orders  
/customers
```

- Cada recurso pode possuir um identificador:

```
/products/10  
/users/25  
/orders/100
```

A URL deve representar principalmente o recurso, enquanto a ação é indicada pelo método HTTP.



Métodos HTTP no REST

- Os principais métodos utilizados são:

- **GET - Consultar recursos**

`GET /products`

`GET /products/10`

- **POST - Criar um novo recurso.**

`POST /products`

- **PUT - Substituir/atualizar integralmente um recurso.**

`PUT /products/10`

- **PATCH - Realizar atualização parcial.**

`PATCH /products/10`

- **DELETE - Excluir um recurso.**

`DELETE /products/10`



URLs REST: Boas Práticas

- Evite colocar ações nas URLs.

- Evitar:

```
/getProducts  
/createProduct  
/deleteProduct/10
```



- Preferir

```
GET /products  
POST /products  
DELETE /products/10
```

- Outras boas práticas:

- Utilizar substantivos
- Manter padrão consistente
- Evitar URLs excessivamente longas
- Utilizar parâmetros para filtros
- Utilizar códigos HTTP corretamente

- Exemplo:

```
GET /products?category=hardware
```




Códigos HTTP em APIs REST

- A API deve informar corretamente o resultado da operação.
- **2xx – Sucesso**
 - 200 OK
 - 201 Created
 - 204 No Content
- **4xx – Erro do cliente**
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found
 - 409 Conflict
- **5xx – Erro do servidor**
 - 500 Internal Server Error
 - 503 Service Unavailable

- **Em GO**

```
w.WriteHeader(http.StatusCreated)
```

- **OU**

```
http.Error(  
    w,  
    "Produto não encontrado",  
    http.StatusNotFound,  
)
```

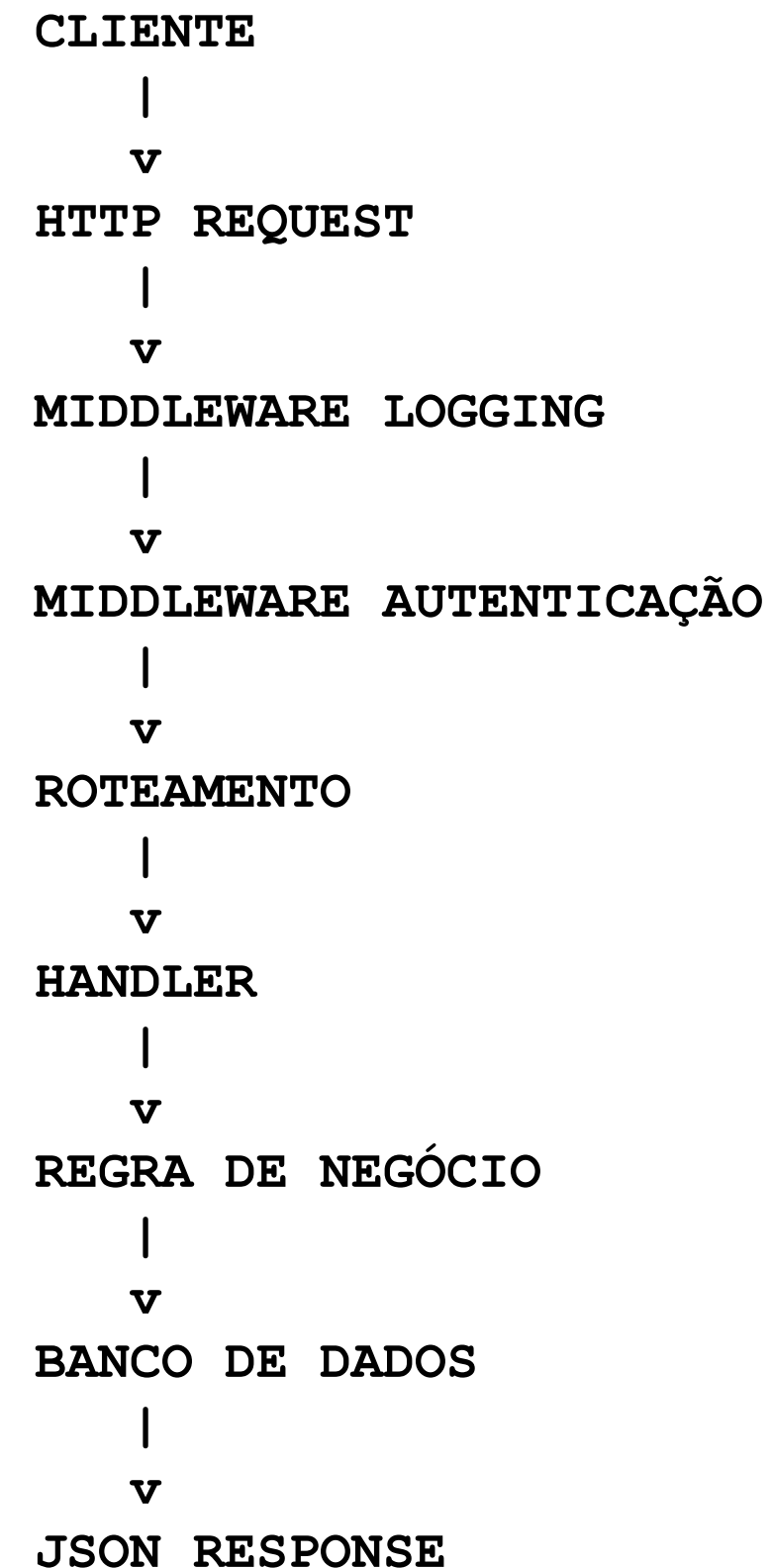


Exemplo de Arquitetura REST em Go

- Uma API pode seguir o seguinte fluxo:

- Exemplo de endpoints:

```
GET    /products
GET    /products/{id}
POST   /products
PUT    /products/{id}
DELETE /products/{id}
```

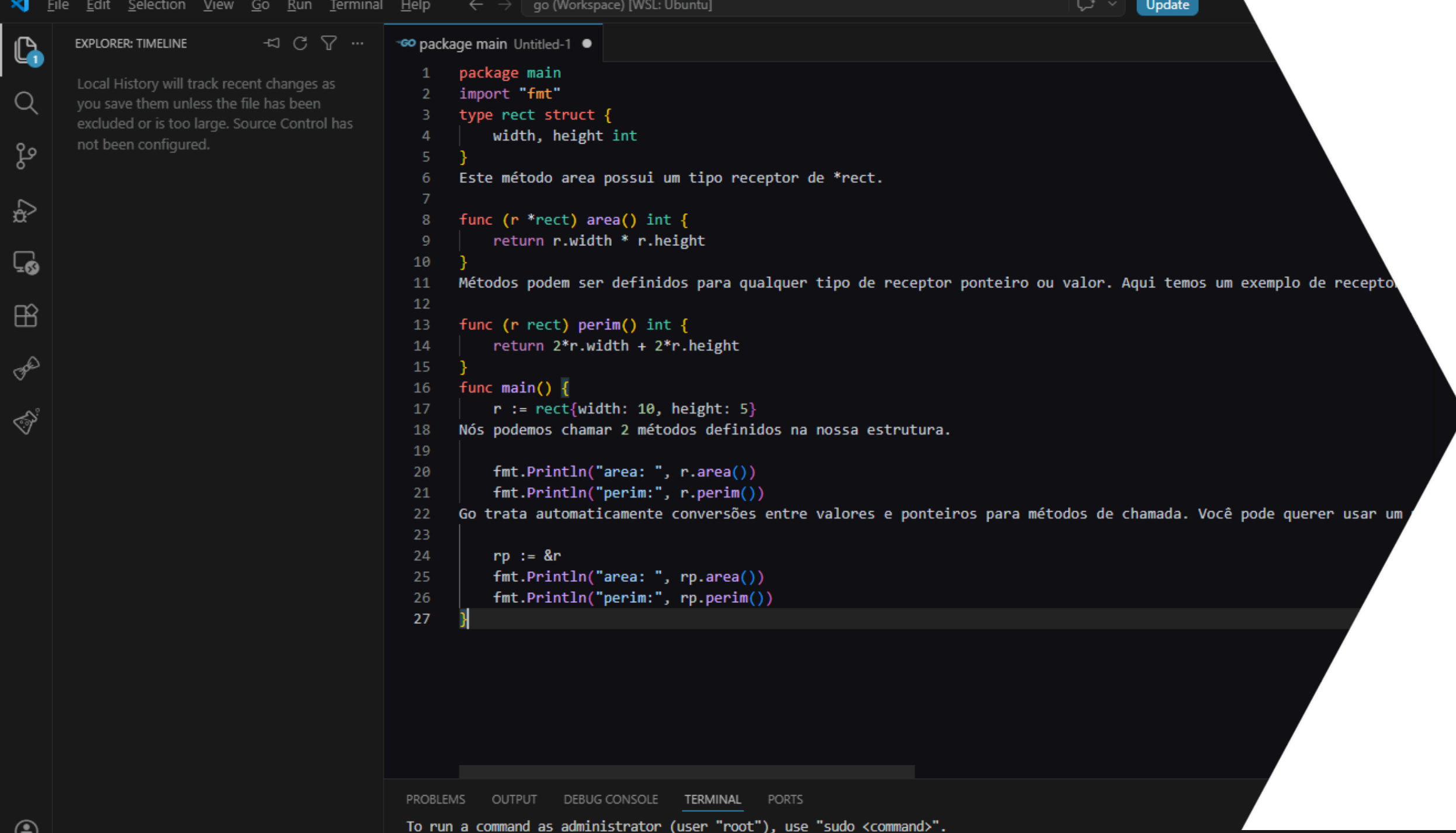




Boas práticas Finais

- **Separe responsabilidades**
- **Utilize Middlewares para funções comuns**
- **Registre as requisições importantes**
- **Proteja endpoints que exigem autenticação**
- **Utilize corretamente métodos e códigos HTTP**
- **Mantenha URLs simples e consistentes**
- **Retorne JSON padronizado**
- **Nunca armazene senhas diretamente no código**
- **Utilize HTTPS em produção**

Resultado: uma API Go mais organizada, segura, escalável e fácil de manter.



```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Middlewares e Boas Práticas

REST em Go